Part 9: Static and Continuous Batching

Companion to Part 9: Static and continuous batching.

These notes explain how several requests share one model while keeping their histories separate. They also follow the repository beyond the original batching implementation into its current shared page pool and direct paged attention.

Read Part 8: Prefill, Decode, and the KV Cache first if the distinction between a processed token and a predicted token is new.

1. Which version are we explaining?

The batching implementation is recorded in commit `cfabb65` (`batching code`). That version has two sequence slots, `BATCH_SIZE = 2`, with a contiguous K/V allocation for each slot. It introduces packed decode rows, shared scratch buffers, static batching, continuous batching, and request completion handling.

The current implementation keeps those ideas but extends the memory system:

Original batching implementation	Current repository
Fixed two-slot scheduler	Runtime slot count derived from the workload and pool
Full contiguous cache allocated per slot	One fixed-size shared pool of physical KV pages
Per-request dense decode attention	Direct paged decode attention with online softmax
Slot availability controls admission	Slot availability and completion-budget credits control admission

Later implementation commits include `fa83cb7`, `fd823b9`, and `313999c`. The paging sections below describe current code additions, rather than claiming that every feature was already present at the Part 9 batching milestone.

The sources used here are the README, the batching and paging sections of the working notes, main.cpp, kernels.cu, and kernels.cuh. The explanation follows the checked-in implementation; it is not a GPU benchmark or a claim that reference-model correctness has been validated in this session.

2. Why batch decode at all?

After prefill, a single request usually contributes one token to each decode forward pass. Multiplying one hidden vector by a large weight matrix can leave GPU resources underused, while another request waits for its turn.

Batching places several independent token rows into the same operations. A shared weight matrix can serve all those rows, and the operation's launch overhead is spread over more useful work. The model weights are loaded once; we do not load another model for every user.

Keep three performance measurements separate:

- **Time to first token:** queue wait, prompt processing, and first prediction.
- **Time between tokens:** the gap between successive predictions for one user.
- **Throughput:** completed requests or generated tokens per unit of time.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For an illustrative calculation, a 10 ms decode step producing one token gives 100 tokens/second. A 12 ms step producing two tokens gives about 167 tokens/second. Total token throughput improved, although each continuing request now waits 12 ms between those decode predictions. These are made-up numbers to explain the tradeoff, not measurements from this engine.

Batching does not make the dependency between one request's future tokens vanish. To predict token E after token D, that request still needs D's forward pass. The parallelism comes from doing useful work for other requests alongside it.

3. Batch size is different from sequence length

During prefill, `token_count = T` means T positions from one prompt. Dense causal attention connects those positions according to the prompt's causal mask.

During decode, `token_count = B` means one current token from each of B independent requests. Their prompt lengths and cache lengths can all differ.

Suppose request A has processed `[A, B, C]`, and request B has processed `[J, K, L, M, N]`. Their latest predictions are D and O:

```
Decode row 0: token D, request A, existing cache length 3
Decode row 1: token O, request B, existing cache length 5

token_ids    = [D, O]
position_ids = [3, 5]
```

After embedding gather, the shared activation matrix is `[2, 2048]`. The rows are two requests, rather than two adjacent positions in one conversation. We must never run a two-row causal attention calculation that lets O read D as though they belonged to the same sequence.

Shapes through one decode layer

The fixed model constants currently are:

Quantity	Value
Transformer layers	16
Hidden width	2048
Intermediate MLP width	8192
Query heads	32
KV heads	8
Head width	64
Vocabulary	128256
Context limit	2048 token positions

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

With B active requests, the principal tensor shapes are:

Operation or tensor	Shape
Hidden state and normalized hidden state	`[B, 2048]`
Q projection	`[B, 2048] = [B, 32, 64]`
New K and new V projections, each	`[B, 512] = [B, 8, 64]`
Attention output and output projection	`[B, 2048]`
Gate and up projections, each	`[B, 8192]`
MLP down projection	`[B, 2048]`
Final logits	`[B, 128256]`, stored as FP32
Greedy predictions	`[B]` token IDs

`projectBf16RowMajor` in `main.cpp` multiplies every input row by the same weights. Its cuBLAS transpose flags account for cuBLAS's column-major view of the row-major buffers; batching does not require transposing the input on GPU.

RMSNorm reduces across a row's hidden features. SwiGLU acts on corresponding elements of each row. Residual addition combines corresponding rows. None of these operations mixes the contents of different requests.

4. Follow one batched decode step

The entry point is `decode` in main.cpp. For our D/O example, one iteration performs the following work:

1. Validate each cache and record positions `[3, 5]` from its current length.
2. Ensure pages exist for prefixes of lengths `[4, 6]` in every layer.
3. Upload block tables in exactly the same packed order as `[D, O]`.
4. Gather both embeddings into the `[2, 2048]` activation buffer.
5. Run all 16 transformer layers in order.
6. Apply final RMSNorm, the LM head, and a separate argmax for each row.
7. Return the predictions and increment each cache length once.

Within each layer, Q/K/V projections run over both rows together. RoPE rotates D's Q/K at absolute position 3 and O's Q/K at absolute position 5. Those positions stay the same through all 16 layers; a layer is not another token position.

The new K/V rows are then copied to their owning sequence's pages. Cached keys already have RoPE applied. Old keys must not be rotated again, and V is not rotated. Attention includes the newly appended token:

```
D's query reads request A positions 0..3: A, B, C, D
O's query reads request B positions 0..5: J, K, L, M, N, O
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Current decode launches paged attention for all active rows and heads. Each block follows one row's own table. Output projection and the MLP then process the resulting packed hidden rows together again.

If argmax returns `[E, P]`, the caches contain D and O, while E and P are merely predicted. The lengths become `[4, 6]`. If both requests continue, the next decode call consumes `[E, P]` at positions `[4, 6]`.

5. Request identity, slot, and packed row

These are different kinds of identity, even when their numbers happen to match:

Name	What it identifies	Current representation
Request index	One input job and its output	Index into `requests`
Sequence slot	Active ownership and cache metadata	Index into `caches` and `slot_requests`
Packed batch row	One row in this decode call	Index into `active_tokens` and `active_caches`

Initially, A might occupy slot 0 and B slot 1. If A finishes, only B remains:

```
slot_requests = [-1, B]
active_slots  = [1]
active_tokens = [B's latest prediction]
active_caches = [&caches[1]]

Packed row 0 -> slot 1 -> request B
```

When decode returns row 0's prediction, `active_slots[0]` finds slot 1, and `slot_requests[1]` finds request B. Treating output row 0 as slot 0 would send the token to the wrong owner.

If C later occupies slot 0, the next packed order can be `[C, B]`. B's logical history has not moved; its temporary row number changed because the batch did.

The same order must hold for token IDs, positions, cache pointers, GPU block tables, logits, and returned token IDs. Current `decode` also rejects duplicate cache pointers within one batch and requires every cache to use the same pool.

6. Buffer reuse is about lifetime

`InferenceBuffers` owns reusable GPU scratch and one cuBLAS handle. Prefill allocates scratch when first needed. Later prompts, layers, and decode steps reuse that storage. Reusing storage means calculating fresh results into it; it does not mean reusing another request's hidden representations.

Current capacity fields have different purposes:

- `prefill_capacity`: the largest encoded prompt length in the workload.
- `batch_capacity`: the runtime maximum number of simultaneously fitting requests.
- `token_capacity`: the larger of those two capacities, for shared row buffers.

The scheduler sizes the possible batch by sorting required page budgets and counting the largest group of smallest budgets that fits. This is only a scratch sizing calculation. The actual request queue remains FIFO.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

When a later workload needs larger capacities, `generateRequests` frees the old buffer set before updating its capacities. Changing the recorded capacity without reallocating would let later kernels write beyond an old allocation.

Safe aliases inside a transformer layer

Storage	Earlier contents	Safe later use
`normalized_gpu`	Input RMSNorm output	Attention output, after Q/K/V have read the norm
`projection_gpu`	Q	Attention output projection, after attention has read Q
`projection_gpu`	Attention output projection	MLP down projection, after the first residual reads it
`gate_gpu`	Gate projection	SwiGLU result, after consuming its original gate values

The residual's base hidden state must survive until its update has been added. Likewise, the normalized input must survive both gate and up projections. The rule is to overwrite a value only after its final reader has finished.

The current operations use ordered CUDA/cuBLAS execution and explicit checks; decode synchronizes after each layer. Adding independent streams later would require explicit dependencies before assuming these aliases remain safe.

The KV pool has a different lifetime. It preserves history across decode steps, while scratch can be overwritten during the next operation. Returning a finished request's pages makes them available to another request, but does not erase the bytes. New owners write valid positions before attention is allowed to read them.

Only dense prefill allocates the `[32, T_capacity, T_capacity]` BF16 attention-score scratch. Current paged decode does not use that score matrix, nor does its K/V scratch need room to gather an entire cached history.

7. Completion and the first-token accounting trap

`prefill` already predicts the first output token. `accept_token`, the completion handler inside `generateRequests`, handles predictions from both prefill and decode. This gives them the same stopping and budget rules.

Current stop checks recognize tokenizer EOT, tokenizer EOS, and `<|eom_id|>` (`128008`). A stop token completes the request and is not appended to the visible generated token list. Other predictions are appended before the budget check.

For a three-token prompt and an output budget of three:

Work completed	Prediction	Visible generated tokens	Processed cache length
Prefill A, B, C	D	D	3
Decode D	E	D, E	4
Decode E	F	D, E, F	5

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The request completes after F is predicted. Processing F would only be useful if we wanted to predict another token after it.

Let P be the encoded prompt length and G the allowed output budget. The largest processed prefix before completion is therefore:

```
maximum_cached_tokens = P + G - 1
```

For `G = 1`, prefill can complete the request without any decode call. For an immediate stop prediction, its response can contain zero visible tokens.

`main` first encodes an Instruct chat prompt, including the headers and special tokens. Its P is not the word count or raw text token count. It requires `P < 2048` and computes:

```
G = min(requested_max_new_tokens, 2048 - P)
```

This is the current public context policy. Do not silently change the formula to allow one more returned token just because the final prediction is uncached.

On completion, the scheduler prints the whole decoded response, releases all assigned pages through `freeKvCache`, returns the admission credits, and marks the slot `-1`. Decoding the entire generated byte sequence also lets split UTF-8 token fragments join correctly. The CLI currently prints finished responses; it does not stream each token to a client as it is produced.

8. Static and continuous scheduling

Both policies use the same batched decode. Their difference is when waiting requests are admitted, as implemented in `generateRequests`.

Assume A and B are already prefilled and admitted. A needs two further decode steps, B needs six, and C is waiting. This illustrative schedule assumes enough page credits for C once A completes.

Static batching

The scheduler admits a group, then waits until every member finishes before admitting another group:

```
Decode round      1  2  3  4  5  6
Slot 0            A  A  -  -  -  -
Slot 1            B  B  B  B  B  B
C waiting         C  C  C  C  C  C

After round 6: prefill C, then start the next group.
```

Finished requests leave subsequent packed decode calls. Static batching does not keep running fake rows for them; it simply leaves the freed slot unused until the group drains. That shrinking batch can reduce useful GPU work.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Continuous batching

The scheduler checks admissions between decode iterations, so C can enter after A completes while B keeps its own cache:

```
Decode A+B -> Decode A+B -> A completes
                              |
                           Prefill C   (B waits)
                              |
Decode C+B -> Decode C+B -> Decode C+B -> ...
```

The decode batch membership changes over time. Continuous admission can make better use of available capacity when responses have different lengths.

There is still one serial prefill per newly admitted prompt. A long prefill can delay B's next token, and several admissions can run several prefills before the next decode iteration. The code does not overlap prefill with decode, split prefills into chunks, or run an asynchronous request-serving API.

One scheduler iteration

```
If continuous mode OR every current slot is empty:
    Admit FIFO requests while there are free slots and enough page credits.
    Prefill each admitted prompt; immediately handle its first prediction.

Pack only active slots into token IDs, cache pointers, and row mappings.
If nobody is active and the queue is empty: finish.
Otherwise run one batched decode step.
Map output rows back to requests and handle completions.
Repeat.
```

In continuous mode a request that completes during prefill can let another queued request use that slot immediately. Static mode breaks after one admission attempt per slot; its next group still waits for the active group.

9. The shared pool: memory follows processed history

At the original two-slot milestone, a full contiguous cache costs:

```
Per token, one layer: 2 (K and V) * 512 values * 2 BF16 bytes = 2048 bytes
Per token, 16 layers: 2048 * 16 = 32768 bytes = 32 KiB
Full 2048-token sequence: 2048 * 32 KiB = 64 MiB
Two full sequence allocations: 128 MiB
```

Short requests used only a small portion of those full allocations. Current `KVCachePool` instead allocates one shared 128 MiB pool by default. Adding more input requests does not multiply this pool allocation.

A physical page belongs to one sequence and one layer while assigned. Its layout is K for 16 positions followed by V for the same 16 positions:

```
K half: [16, 512] BF16 = 16384 bytes
V half: [16, 512] BF16 = 16384 bytes
Physical page total  = 32768 bytes = 32 KiB

128 MiB / 32 KiB = 4096 physical pages in the default pool
2048 / 16 = 128 logical pages per sequence, per layer
16 layers * 128 pages = 2048 pages for one full-context cache
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

One 16-token logical span across all 16 layers needs 16 physical pages, or 512 KiB. Do not confuse a 32 KiB physical page with a complete 16-layer span.

`KVBlockTable` holds `[16, 128]` page IDs for each sequence. Unassigned entries are `-1`. For layer 0, a possible mapping is:

```
Logical page 0 -> physical page 5   (positions 0..15)
Logical page 1 -> physical page 1   (positions 16..31)
Logical page 2 -> physical page 7   (positions 32..47)
```

Position 33 is in logical page `33 / 16 = 2`, at offset `33 % 16 = 1`. The table routes it to physical page 7, at that same token offset. Layer 1 has its own mapping and its own values.

This is an application-managed mapping inspired by virtual memory. It does not make GPU memory automatically spill to CPU RAM, and it is not OS page faulting. The pool itself is one contiguous GPU allocation; a sequence's logical history is distributed among pages within it.

10. Admission credits prevent future pool exhaustion

Lazy assignment alone is insufficient. If the scheduler admits requests based only on pages used right now, several growing requests could later need more pages than remain available.

The current scheduler commits enough credits for each admitted request to finish its declared output budget:

```
cached_budget = P + G - 1
pages_per_layer = ceil(cached_budget / 16)
required_pages = pages_per_layer * 16

Admission condition:
required_pages <= pool_page_count - committed_pages
```

For `P = 40` and `G = 25`, the largest processed prefix is 64 tokens. It needs four pages per layer, or 64 physical pages = 2 MiB of credits. Its prefill initially assigns only three pages per layer, or 48 pages = 1.5 MiB. At processed position 48, it receives another page per layer using credits that were already committed. Admission credits and assigned pages are different.

`KVPageAllocator::reserve` counts all missing pages across all layers before changing any table entries. If too few physical pages exist, it returns failure without partially updating that table. Prefill reserves enough for the prompt; decode reserves enough for each current `length + 1`.

When a request ends early, both its assigned pages and its full credit commitment are released. Unused credits were never physical page assignments.

FIFO admission does not skip a large head request to admit smaller requests behind it. This can leave capacity temporarily unused. A request whose complete budget exceeds the whole pool is rejected before generation starts, with a message suggesting a smaller output budget or larger pool.

Within this policy, active requests have room to finish. The scheduler needs no preemption, CPU swapping, or recomputation to resolve later page-boundary growth. The cost is conservative concurrency: a large requested output allowance consumes credits even if the model will probably emit a stop token much earlier.

11. Direct paged attention and online softmax

Paging answers where K/V live. Attention still needs one normalized weighted sum over the sequence's entire valid history. Current decode follows the pages directly instead of gathering the prefix into a contiguous temporary cache.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The implementation is `pagedAttentionKernel` and its `launchPagedAttention` wrapper in kernels.cu. Unlike the other public launchers, this wrapper is currently forward-declared near the top of `main.cpp`, rather than declared in `kernels.cuh`.

Threads, heads, and addresses

The launch uses a grid `[B, 32]` and 64 threads per block:

- `blockIdx.x` selects the packed request row.
- `blockIdx.y` selects one of 32 query heads.
- `threadIdx.x` selects one of the head's 64 components.

Each block handles one query head. GQA maps it to KV head `query_head / 4`: query heads 8, 9, 10, and 11 all use KV head 2. They still have distinct queries and attention outputs.

The valid history length is `position_ids[row] + 1`, including the just-appended token. The last page is read only up to that valid count. No future-token mask is necessary because decode has one query and reads only its existing prefix.

For physical page p, token offset t, KV head h, and component d, element offsets within the BF16 pool are:

```
page_start = p * (2 * 16 * 512)
key_index  = page_start + t * 512 + h * 64 + d
value_index = key_index + (16 * 512)
```

For every cached position, each thread multiplies its Q component by the matching K component. Warp shuffles reduce the two 32-thread halves; shared memory combines them into the 64-component dot product. Dividing by 8 applies the attention scale `1 / sqrt(64)`.

Keep a running normalized sum

For scores `s_i = dot(q, k_i) / 8`, ordinary stable attention computes:

```
m = max(s_i)
output = sum(exp(s_i - m) * v_i) / sum(exp(s_i - m))
```

Online softmax maintains the same information while reading positions one at a time. Let m be the running maximum, l the scaled denominator, and a the scaled weighted-value sum. Initialize `m = -infinity`, `l = 0`, `a = 0`. For a new score s and value vector v:

```
m_new = max(m, s)
correction = exp(m - m_new)
weight = exp(s - m_new)

l = l * correction + weight
a = a * correction + weight * v
m = m_new

After processing all valid positions: output = a / l
```

The weighted sum a has 64 components. Each thread owns one component in an FP32 register; the maximum and denominator are also maintained in FP32. If a larger maximum arrives, both old sums are rescaled onto the new common exponential scale. Rescaling only the denominator would change the result.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For a scalar illustration, use scores `[1, 2, 1]` and values `[10, 20, 40]`:

Position	Running maximum	Correction	New weight	Denominator	Weighted sum
First	1	0	1	1	10
Second	2	0.367879	1	1.367879	23.678794
Third	2	1	0.367879	1.735759	38.393972

The result is approximately `38.393972 / 1.735759 = 22.1194`. It matches normalizing the three weights `[exp(-1), 1, exp(-1)]` together. Real attention runs the value accumulation independently for all 64 components.

Crossing a page boundary changes the K/V address, not the normalization problem. The kernel retains m, l, and a while following the next logical page's mapping. Normalizing each page separately and averaging page outputs would generally be wrong because pages can contribute different total softmax probability mass.

This implementation avoids a decode score buffer and contiguous KV gather. It still visits every valid cached position for each query head. Its two-warp block and serial loop over positions are a teaching implementation, rather than a claim of matching highly tuned production attention kernels.

12. Running the scheduler and reading its statistics

With the dependencies and matching model files described in the README:

```
cmake -S . -B build
cmake --build build -j

# Continuous batching is the default; each request gets up to 64 new tokens.
./build/samir-vllm --continuous --stats "Say hello." 64 "Explain GQA." "What is RoPE?"

# Admit a new group only after the previous group has drained.
./build/samir-vllm --static --stats "Say hello." 64 "Explain GQA." "What is RoPE?"

# Keep the same workload but reduce the shared pool to 8 MiB.
./build/samir-vllm --kv-cache-mib 8 --stats "Say hello." 64 "Explain GQA." "What is RoPE?"
```

Flags precede positional arguments. The first positional argument is a prompt, the second is the output budget, and any remaining arguments are more prompts. That one requested budget applies to every request before context clamping. Use `--` before a prompt beginning with `--` so it is not parsed as an option.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

`--stats` prints a JSON object after successful generation:

Field	Meaning
`pool_bytes`	Bytes allocated for the shared KV data pool
`pool_pages`	Number of physical 32 KiB pages in that pool
`peak_active`	Highest simultaneous admitted request count, including prefill
`max_decode_batch`	Largest actual number of rows passed to decode
`peak_pages`	Highest number of physically assigned pages observed
`admission_waits`	Failed FIFO-head admission attempts because credits did not fit
`final_free_pages`	Physical pages returned to the allocator at completion

`peak_active` can exceed `max_decode_batch`: an admitted request can finish on its prefill prediction and never enter a decode batch. `admission_waits` is a count of attempts, not elapsed waiting time or a count of distinct blocked jobs.

The final free count should equal the pool page count. The engine checks that all physical pages and committed credits were returned before reporting success. The pool figure excludes model weights, scratch, tables, and other GPU memory.

13. Correctness and performance checks worth making

Reasonable-looking text is useful feedback, but it cannot prove that packed rows use the right cache. On a supported target GPU, compare intermediate tensors and logits with an independent implementation using the exact same checkpoint and encoded token IDs.

Exercise different prompt lengths, a request finishing while another continues, replacement of a low-numbered slot, and appends across positions 15/16 and 31/32. Also cover a one-token output budget, immediate stop, early page release, FIFO admission waits, and rejection of a budget larger than the entire pool.

For a paged-attention reference, gather only the valid K/V prefix and compare its dense FP32-softmax result with the direct paged result within a justified tolerance. Match GQA head mapping, RoPE positions, scale, and inclusion of the current token. `gatherKvCache` remains in `main.cpp` as a dense reference helper; current decode does not call it.

BF16 projections can round differently at different batch sizes. When two vocabulary logits nearly tie, a small difference can change greedy argmax, and the resulting generations can diverge. The argmax kernel's smallest-ID tie rule only resolves exactly equal logits; it cannot make near ties batch invariant. Inspect numerical error and the top-logit gap when comparing runs.

Measure queue delay, prompt time, inter-token gaps, and total throughput separately. A larger decode batch can improve total tokens/second while serial prefill increases one continuing user's next-token delay. These notes provide the code model and suggested checks, not measured performance results.

14. Reading the code with a purpose

Follow this route through main.cpp:

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

1. Read `Request`, `KVCache`, `KVBlockTable`, and `InferenceBuffers` to identify ownership.
2. Read `main` to see chat formatting and output-budget clamping.
3. Read the start of `generateRequests` for credit accounting and runtime sizing.
4. Follow `accept_token` for completion and page release.
5. Follow the admission loop, then `active_slots`, then the returned-row mapping.
6. Read `decode` to connect packed rows to positions and uploaded tables.
7. Read `appendKvCache` and `KVPageAllocator::reserve` for physical storage changes.
8. Read `pagedAttentionKernel` in kernels.cu for direct page traversal.

The central invariant is that batching shares model computation while every row retains its own request identity, absolute positions, and valid history. The scheduler decides which requests may run; the page allocator supplies their storage; attention follows each row's mappings to read that request's context.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.